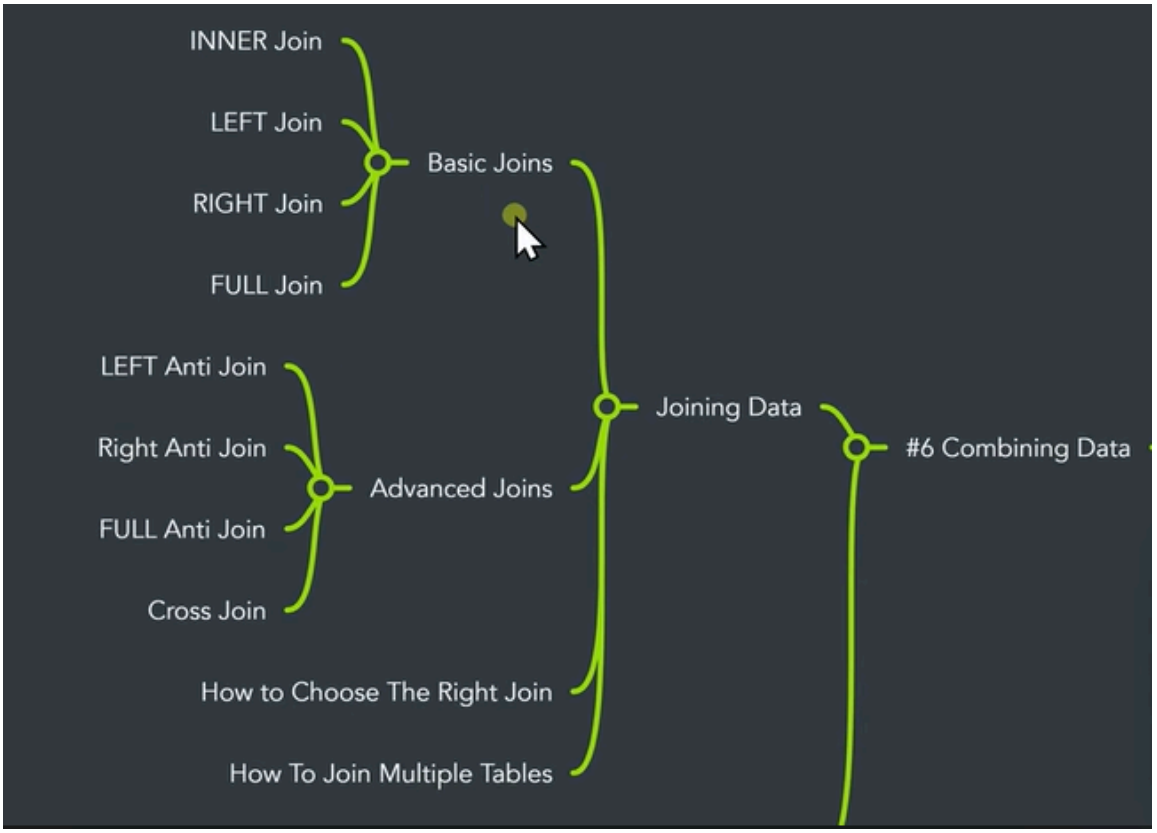
Combining Data



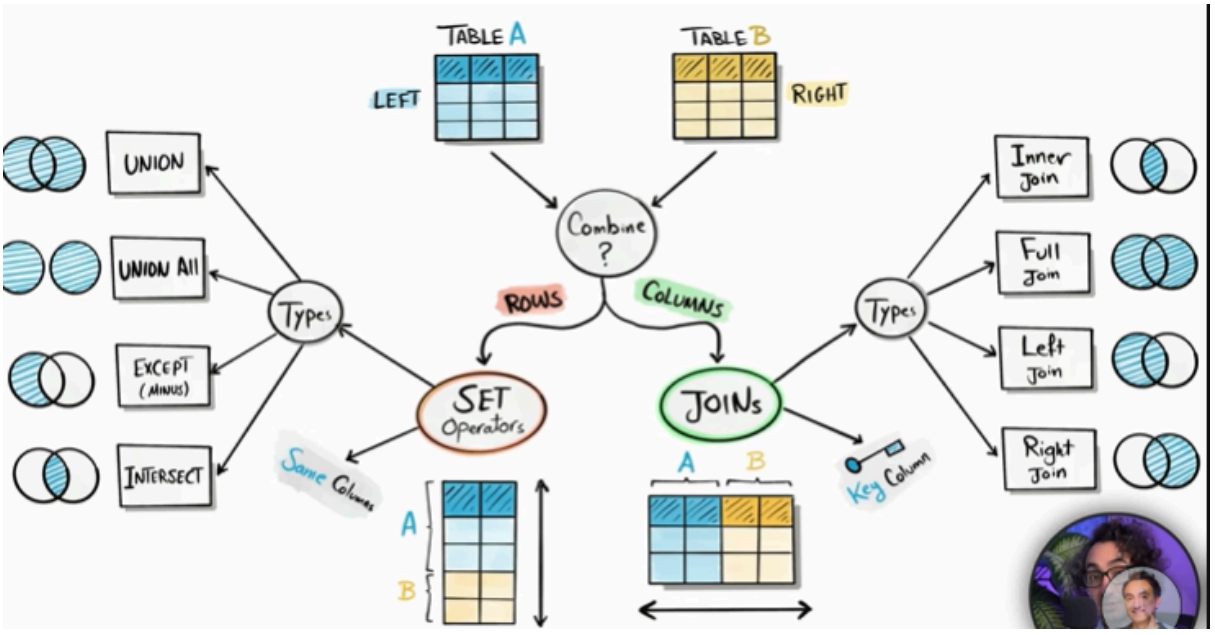
1. Joining Data



2. Set Operations



1. Difference between Set,Join

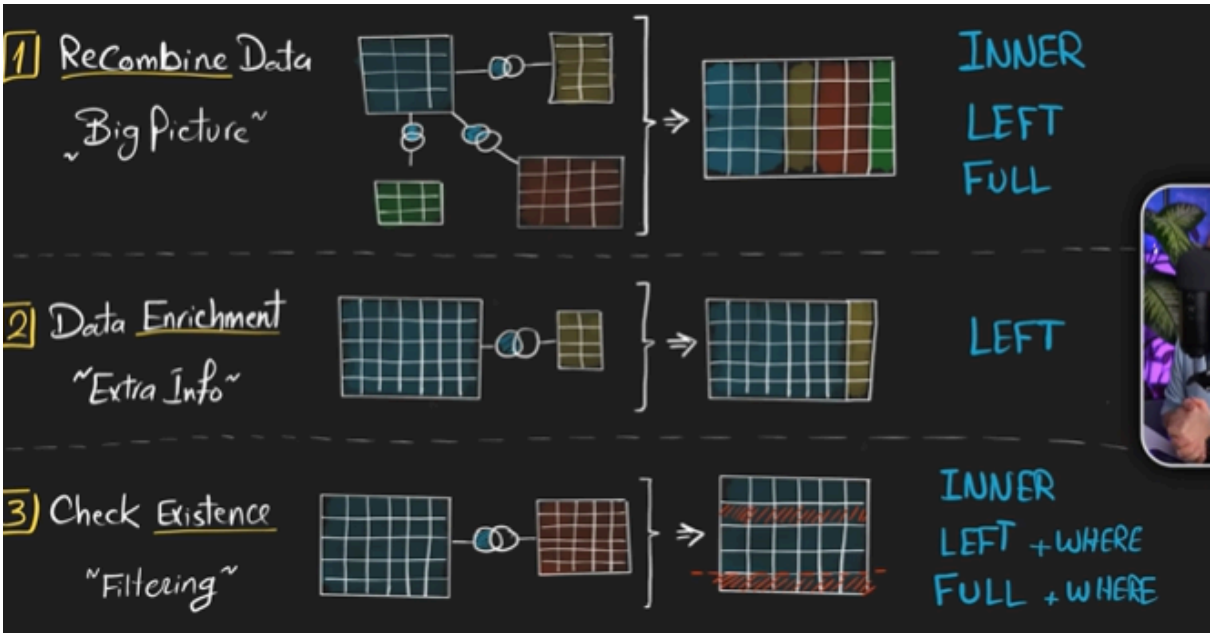


1. In Set the combining of two tables occur in row wise → means the first table and second table are jointed along the rows
2. For Set , both the joining tables must have the same columns→same data-type → same schema
3. the data size increases row wise

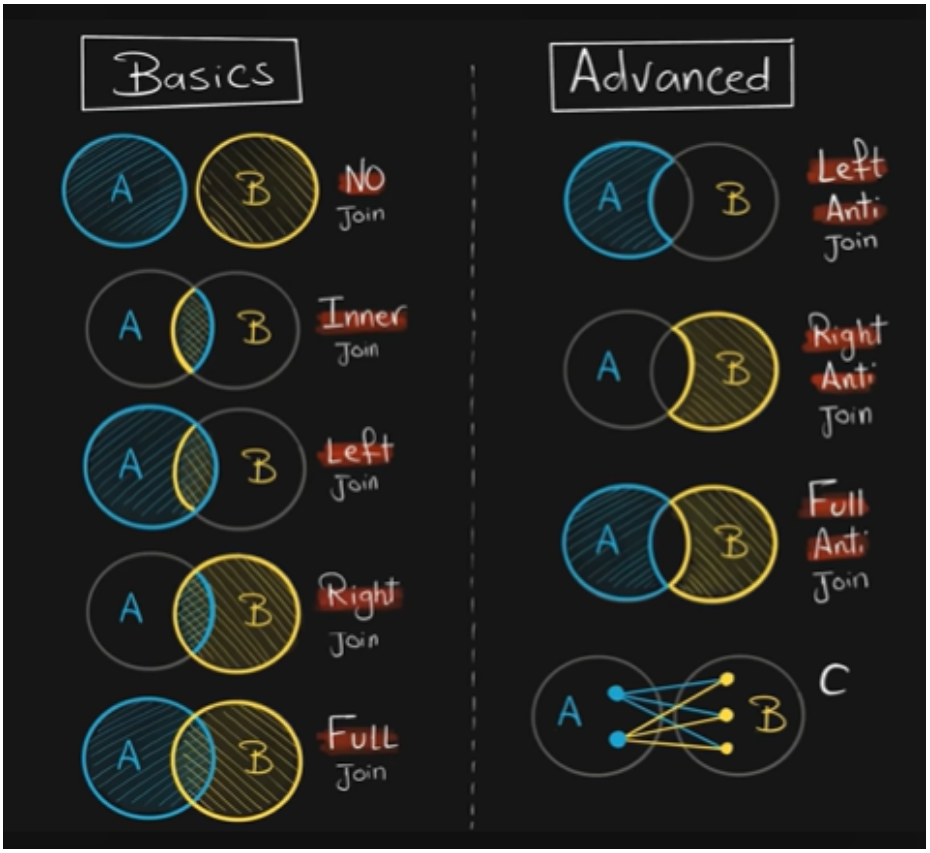
1. In Joins the two tables will combined side-by-side

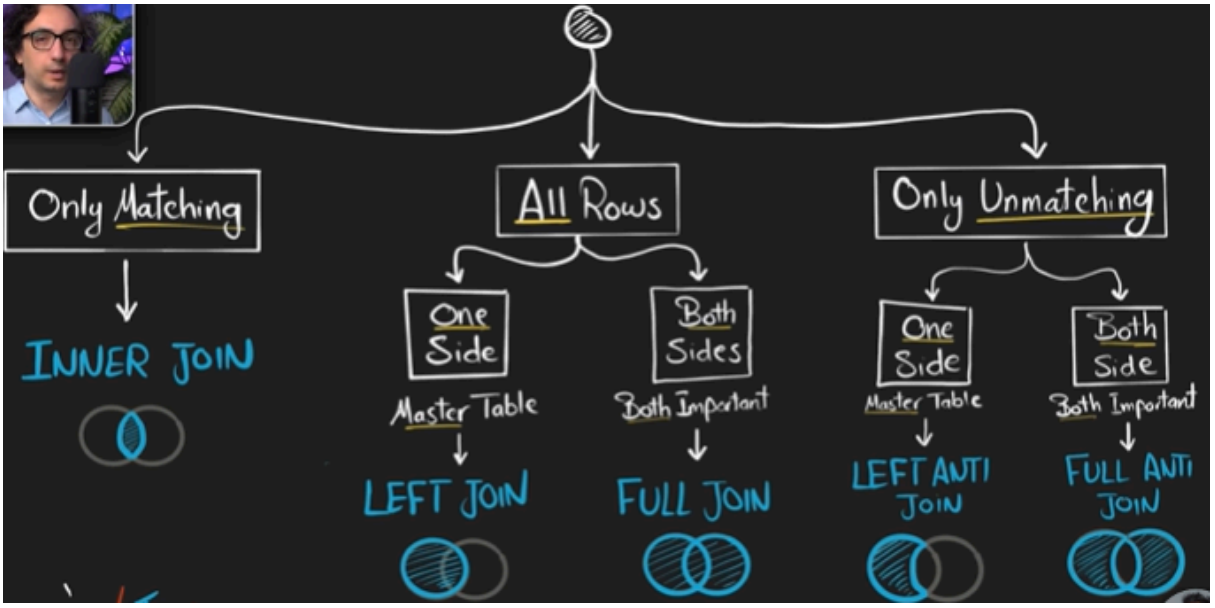
2. If no value is there then NULL will be inserted there .

2. Joins Use



3. Joins





1. No Join

1. Here directly we can select the two seperate tables enough → no joining

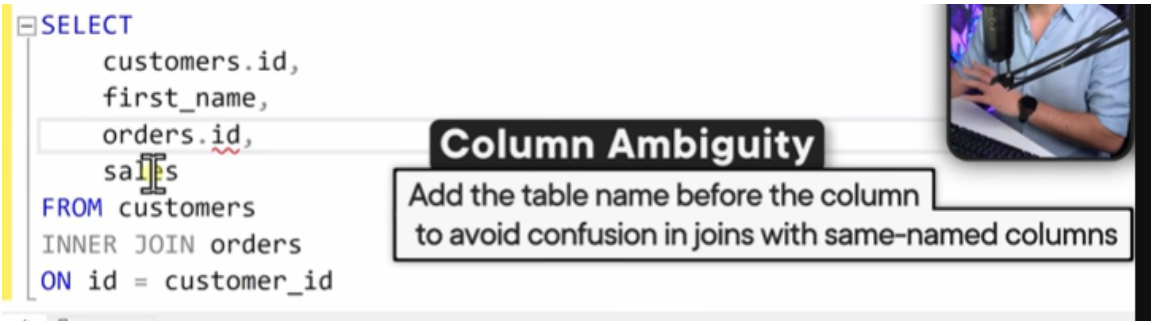
```
SELECT *
FROM customers;

SELECT *
FROM orders;
```

2. They will be retrieved seperately without combining them

2. Inner Join (Default join)

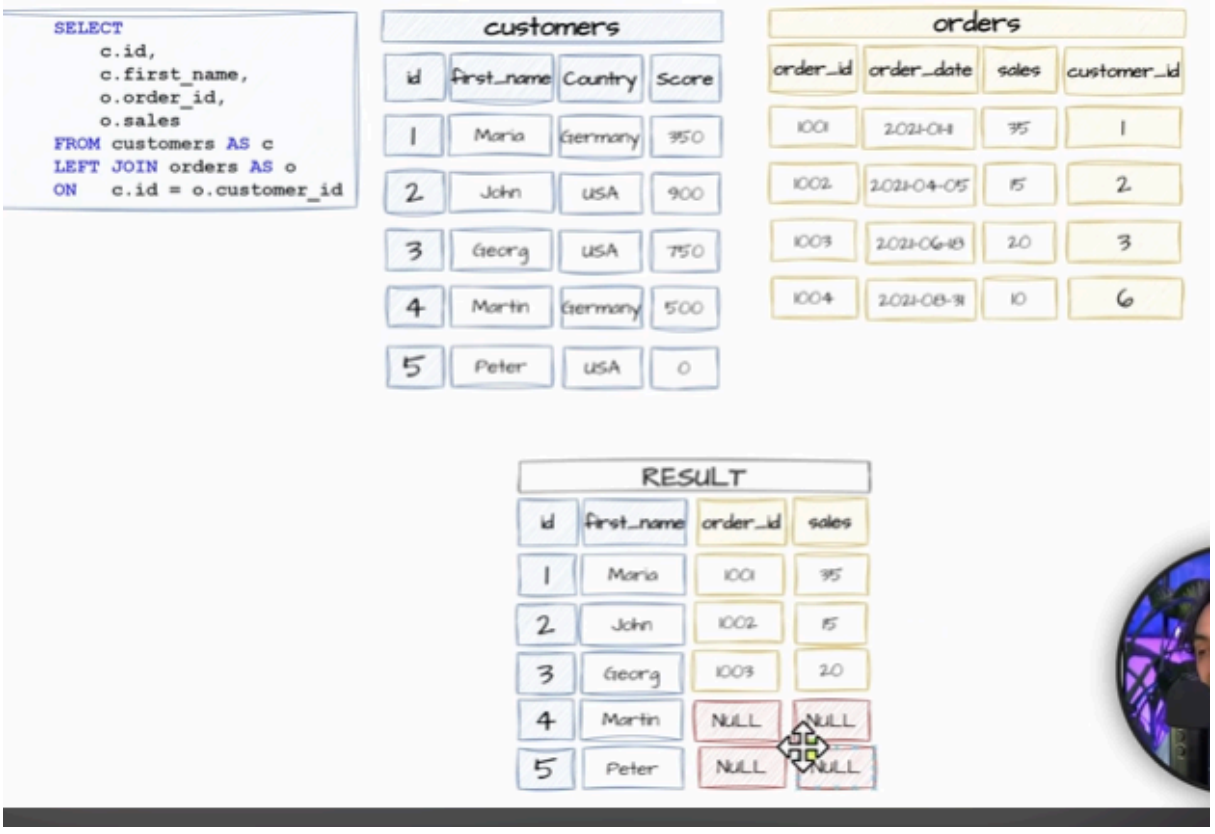
1. It returns only Matching Rows from both Tables
2. we need to join them using some condition , using where clause → better use Primary,Foriegn Key Matching



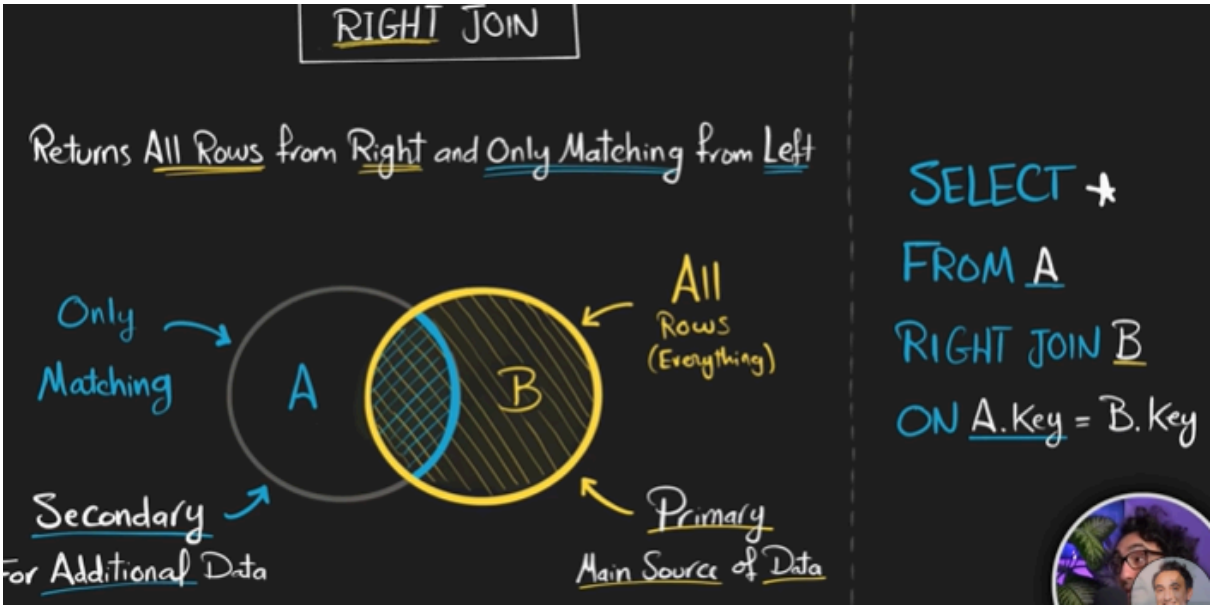
```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM customers AS c
INNER JOIN orders AS o
ON c.id = o.customer_id
```

3. Left Join

1. Returns all rows from Left and only matching from Right
2. Left table will be there and will be combined to right table only where condition satistifies , for remaining rowns of table 1 there will be NULL values of table 2



4. Right Join



1. Here right table will be completely there and left table values those matches the condition will only be there
2. it can be done with Left-Join also but we need to reverse the table order
    a. SELECT *
    b. FROM B
    c. LEFT JOIN A
    d. O A.key=B.key

    Here B table completely there and A table matched rows only be there === Right join

5. Full Join

1. Combining the two tables
2. table A combines with Table B with condition satisfied rows
3. and condition dissatified rows of left table , right table will also be there but with null values

```sql
SELECT
    c.id,
    c.first_name,
    o.order_id,
    o.sales
FROM customers AS c
FULL JOIN orders AS o
ON c.id = o.customer_id
```

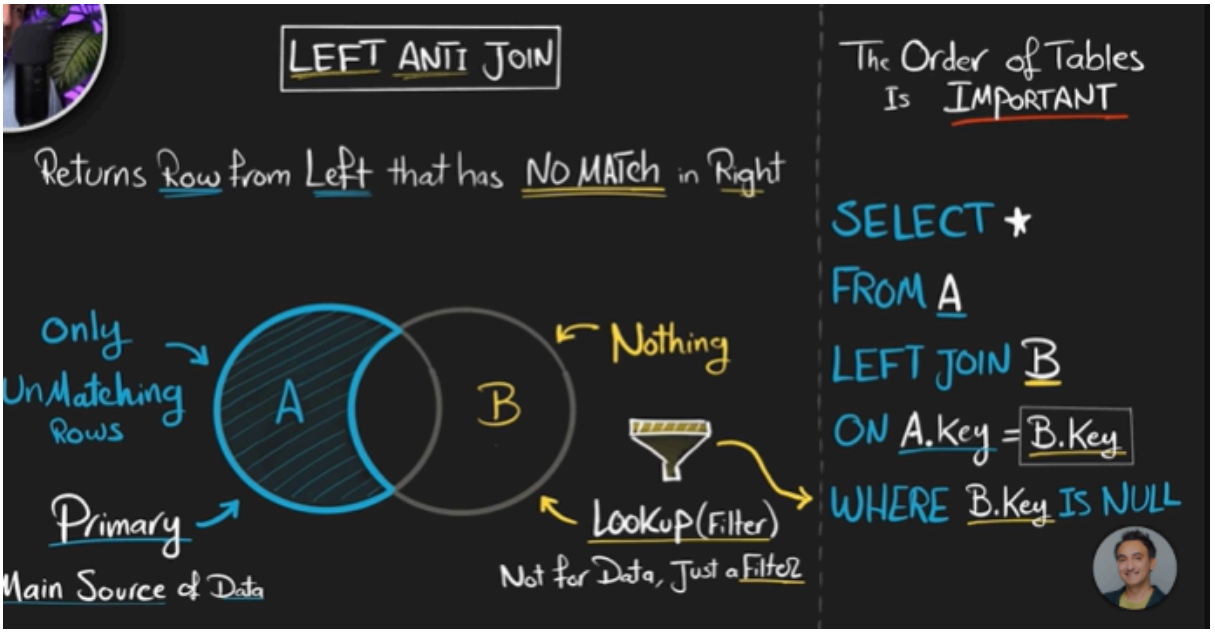
sults Messages

id	first_name	order_id	sales
1	Maria	1001	35
2	John	1002	15
3	Georg	1003	20
4	Martin	NULL	NULL
5	Peter	NULL	NULL
NULL	NULL	1004	10

4. Advanced Joins

6. Left ANTI Join

1. Directly we not have the clause for this , we need to use other Joins to do



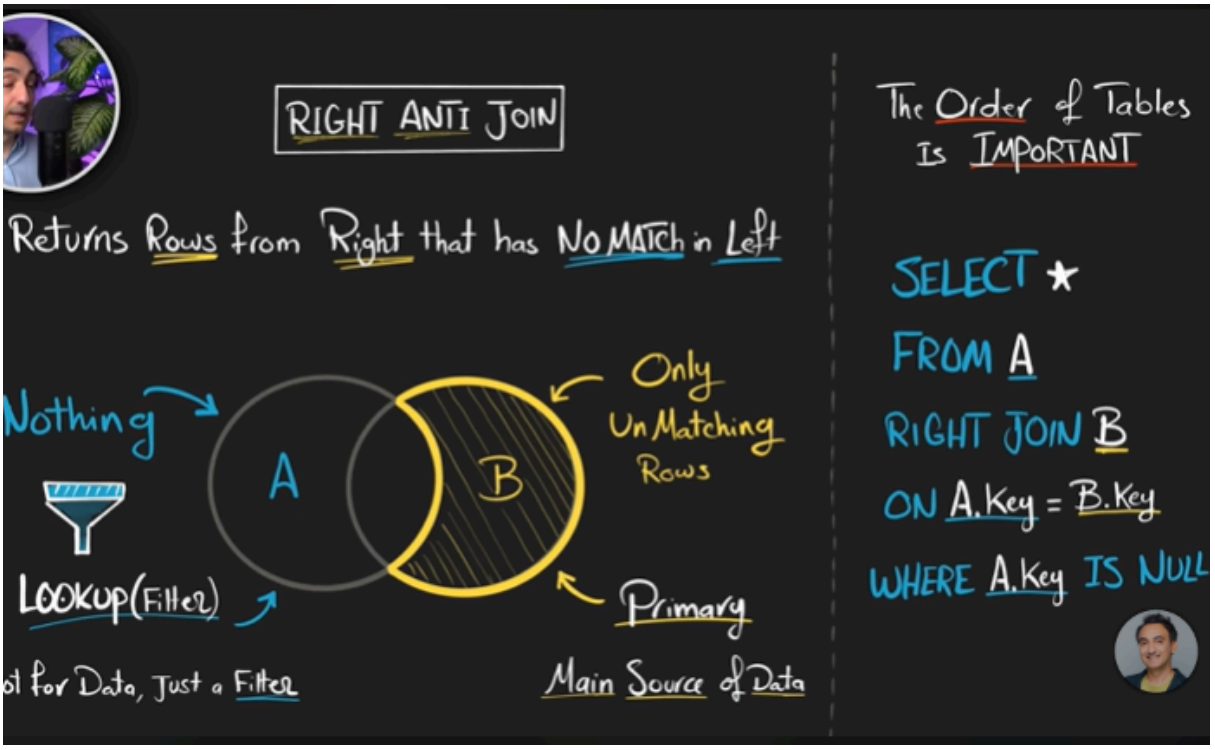
1. When left join used all the rows will appear . and right table is joined to left table if condition satisfies ,and remaining rows will be null⇒ says if null is

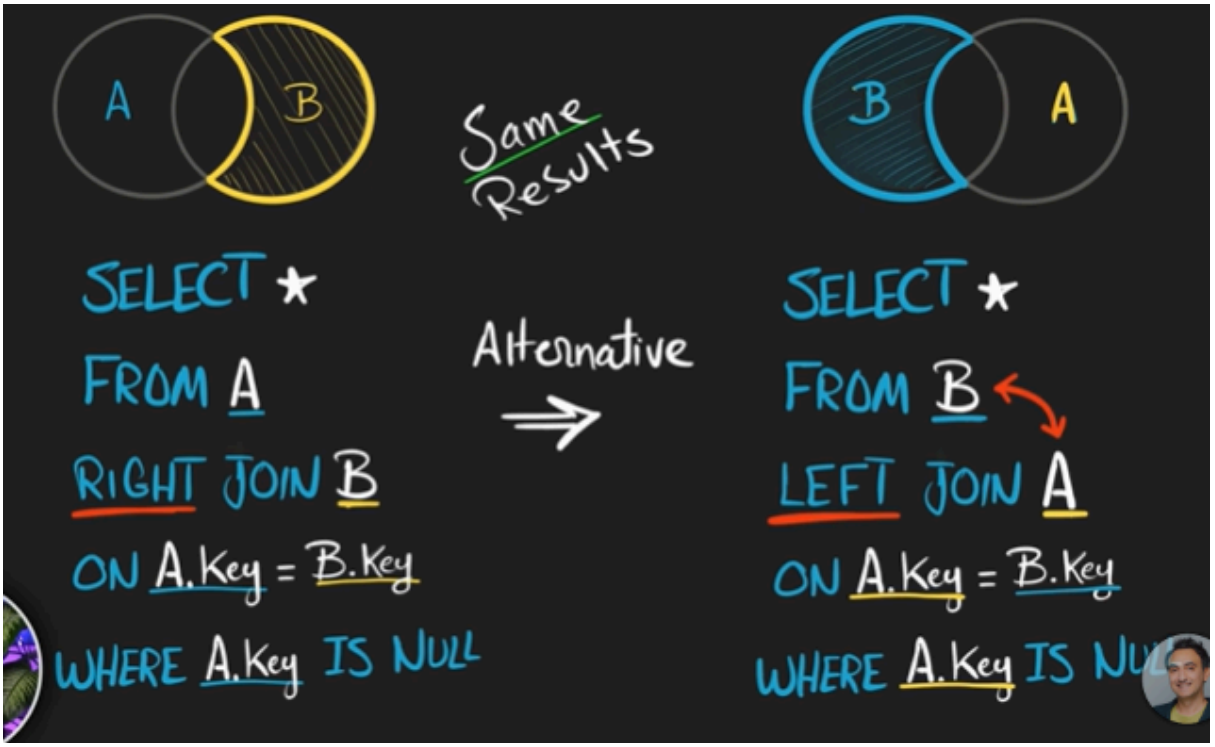
there then those rows donot match condition with rows in right table → those values are required

2. In left but not in right

7. Right ANTI Join

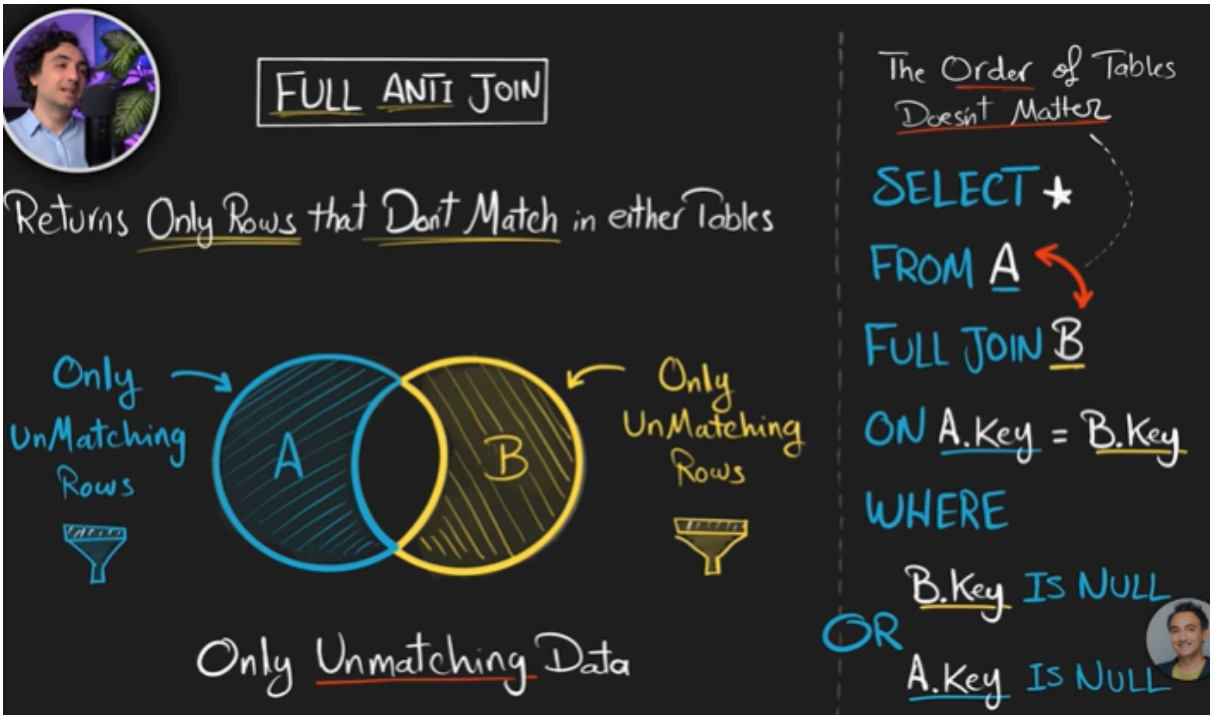
1. Similar to Left ANTI Join but reversing the tables



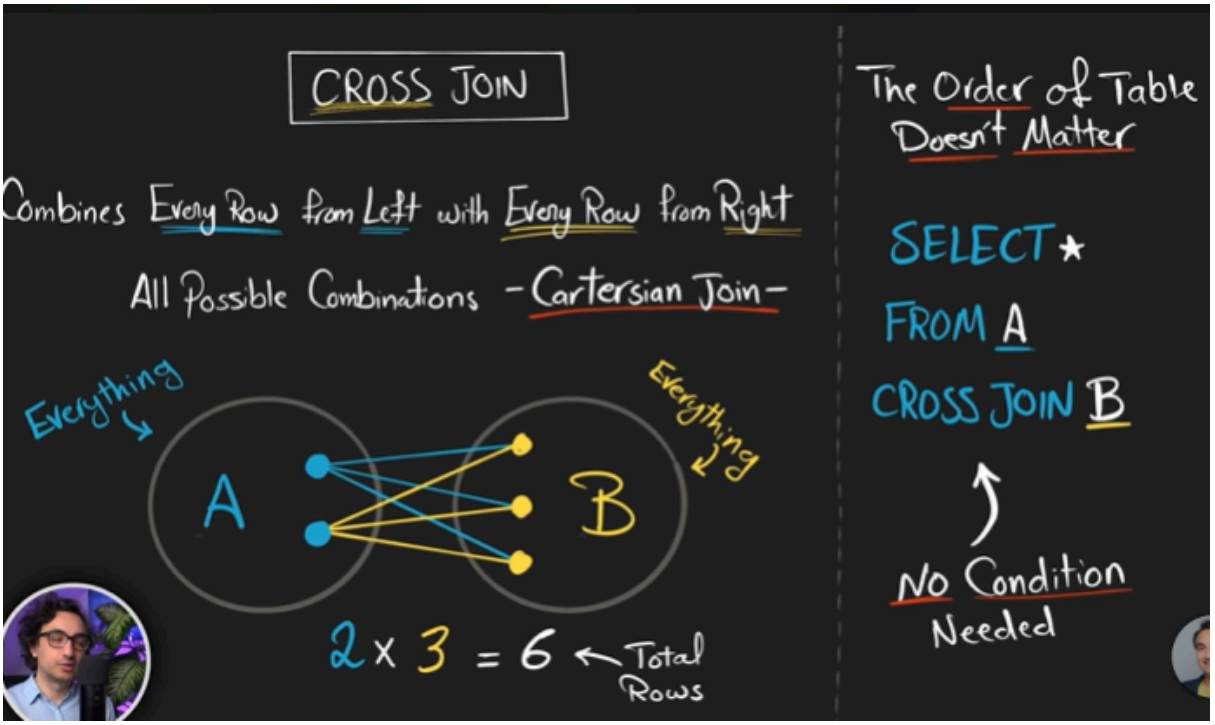


8. Full ANTI Join

1. Returns only rows which donot match in either Tables



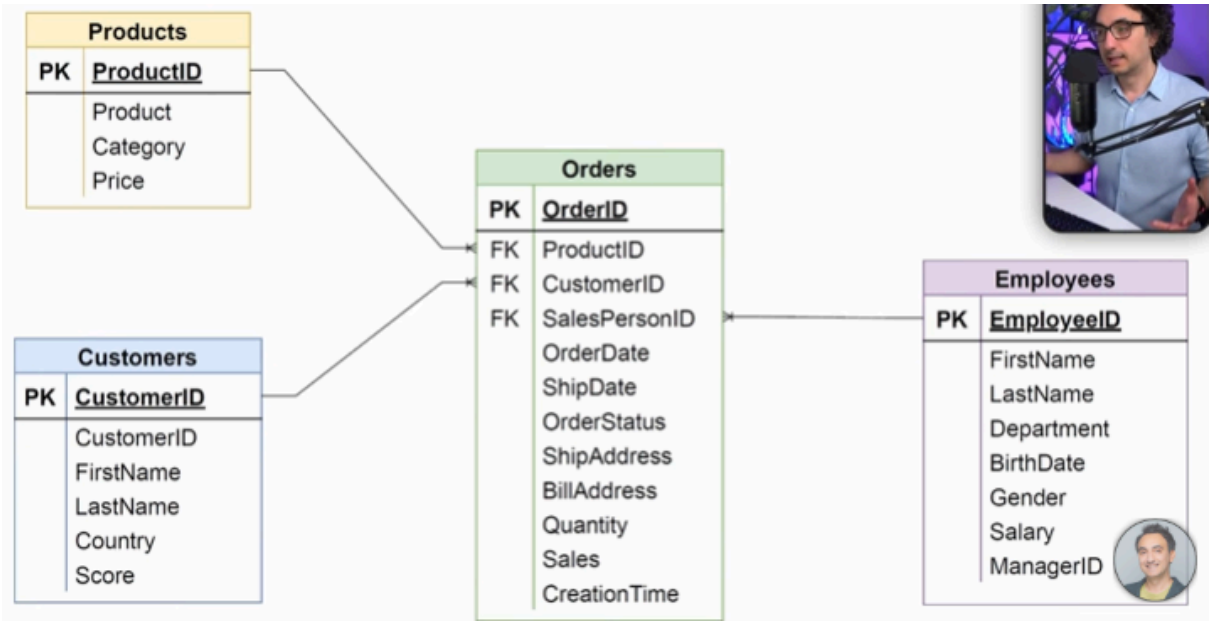
9. Cross Join



1. returns all the possible combinations of joins

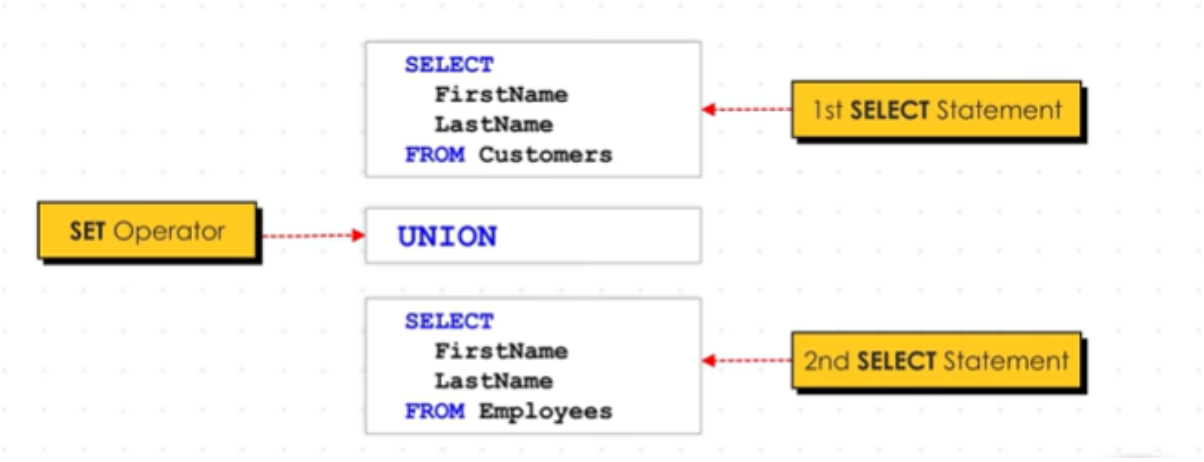
10. Multi Join Table

1. to Join multiple tables we can use this ..
2. It is nothing but using the normal joins multiple times.



```sql
SELECT
    o.OrderID,
    o.Sales,
    c.FirstName AS CustomerFirstName,
    c.LastName AS CustomerLastName,
    p.Product AS ProductName,
    p.Price,
    e.FirstName AS EmployeeFirstName,
    e.LastName AS EmployeeLastName
FROM Sales.Orders AS o
LEFT JOIN Sales.Customers AS c
ON o.CustomerID = c.CustomerID
LEFT JOIN Sales.Products AS p
ON o.ProductID = p.ProductID
LEFT JOIN Sales.Employees AS e
ON o.SalesPersonID = e.EmployeeID
```

5. Set Operators



1. For set Operators it is better to use static variables to know from which table the data came from



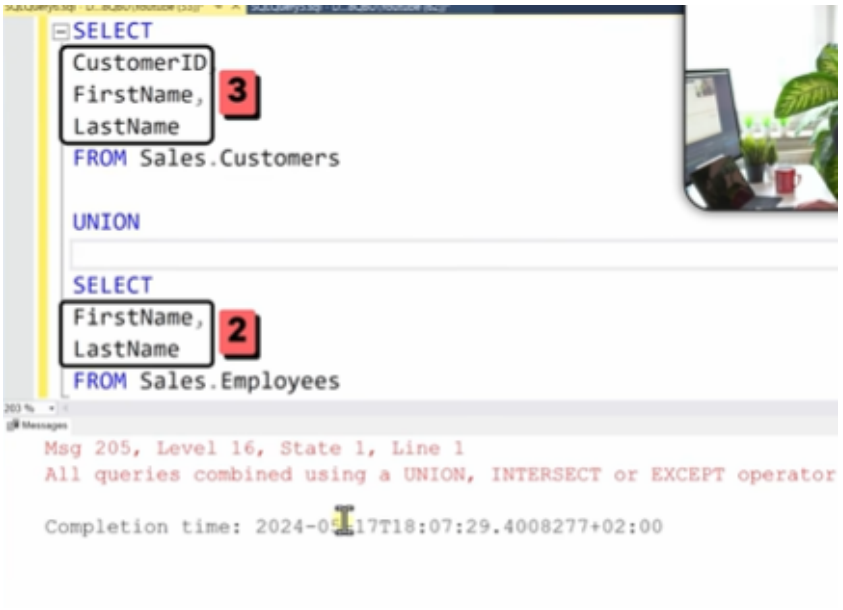
6. RULES for Set

1. Rule-1

1. Set Operator can be used almost in all clauses → where/join/group-by/having
2. But for Order-By is allowed only once at the end of the query (means after combining them)

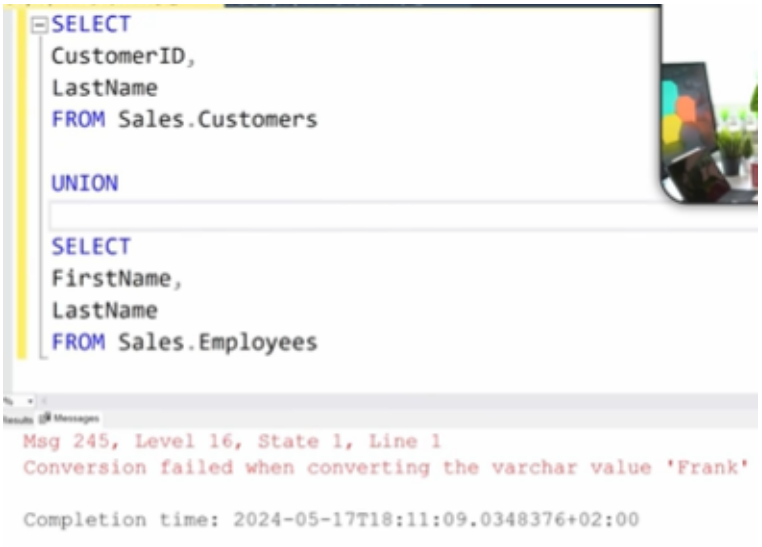
2. Rule-2

1. The number of columns in each query must be the same
2. If not then error will occur



3. Rule-3

1. data Types of columns in each query must be compatible
2. means their respective data-types must be equal



4. Rule-4

1. the order of the columns in each query must be the same
2. SELECT f_name,state from table1 UNION SELECT country,l_name ⇒ here for f_name column the country is attached and for state→ l_name is attached ⇒ order matters

5. Column Aliases

1. the column names in the result set are determined by the column names specified in the first query



2. so to change column names use alias for first select query

7. Set Operators

1. UNION

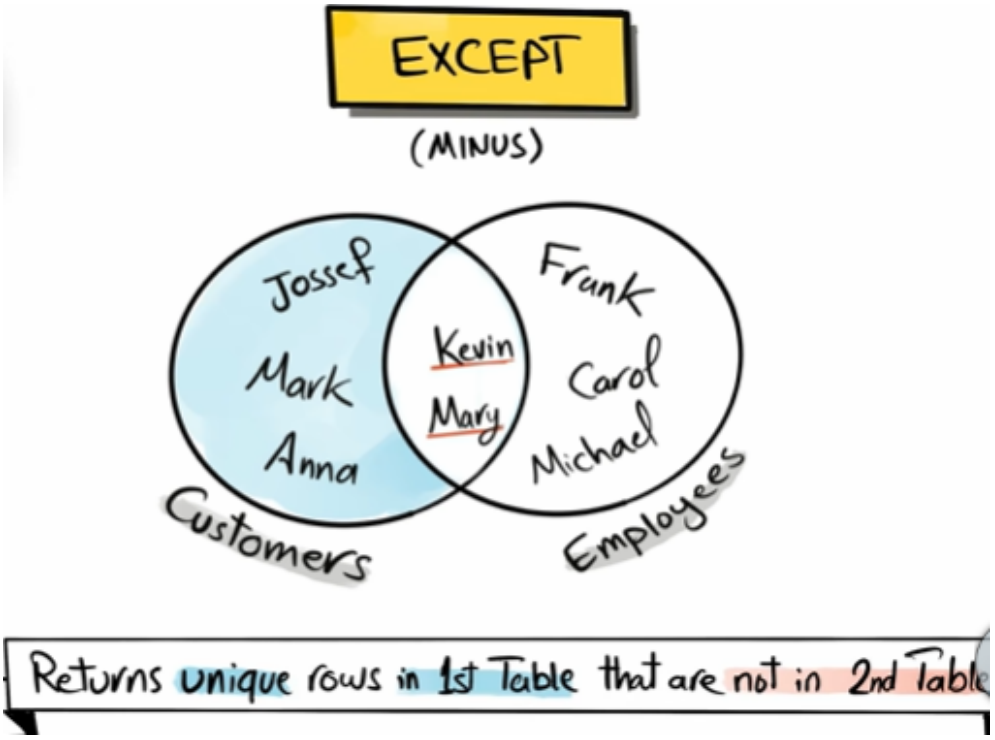
1. Returns all distinct rows from both tables
2. It removes the duplicate rows from the result
3. It only checks the duplicates for the selected columns
4. Order of tables donot matter

2. UNION ALL

1. Returns all rows from both queries , including duplicates
2. UNION ALL is generally faster than UNION
3. Order of tables donot matter

3. EXCEPT

1. Returns all distinct rows from the first query that are not found in the second query
2. Like LEFT_ANTI_JOIN
3. so we can say it is the only one where the order of queries matter



4. if tables changed then o/p will also change

4. Intersect

1. Here we get the intersect part of two tables
2. similar to Inner Join

i 'm excited about this role because it matches well with my interest in building real-world applications using React Native and modern tech. i have already worked on projects like ecommerce , AI-based trip planner ,chat app and many other . even though if i build projects or work on aiml or generative ai any , i prefer to use frontend react or reactnative , instead of streamlit or django libraries in python , because it makes easy and can build best User interface as they offer best functionalities hooks etc, to build . so i enjoy building projects with them .
i am thinking of to work under guidance of the best team and best company , because that helps me to learn more skills in react or react native or ts , js many other , and also helps me to build real products and scaling them , i think this is the best opportunity to move with .